University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	**Course Code:** CMPE-112L
Assignment Type: Lab Report	**Dated:** July 1, 2026
Semester: 1st Semester	**Session:** Spring 2026
Lab/Project/Assignment #: 10	**CLOs to be covered:** CLO2
Lab Title: Recursion	**Teacher Name:** Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	**Level2**	**Level3**	**Level4**	**Level5**	**Level6**
(5)						
					Total	/5

Rubrics for Current Lab Evaluation

Scale	**Marks**	**Level**	**Rubric**

Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 10

Lab Goals / Objectives:

• Understand the concept of recursion in Python

• Learn the components of a recursive function

• Differentiate between iterative and recursive approaches

• Implement recursive solutions for mathematical problems

• Understand the call stack and recursion depth

• Learn about base cases and recursive cases

• Apply recursion to solve the Fibonacci sequence problem

Equipment Required:

• Computer System with Python 3.x installed

• Text Editor or IDE (PyCharm, VS Code, or IDLE)

• Operating System: Windows / Linux / macOS

Theory:

1. What is Recursion?

Recursion is a programming technique where a function calls itself to solve a problem by breaking it down into smaller, similar subproblems. A recursive function solves a problem by solving smaller instances of the same problem.

2. Components of a Recursive Function

Every recursive function must have two essential components:

• Base Case: The condition that stops the recursion. It provides a direct answer without making further recursive calls.

• Recursive Case: The part where the function calls itself with a modified (usually smaller) version of the original problem.

3. How Recursion Works

When a recursive function is called, each call is placed on the call stack. The function keeps calling itself until the base case is reached. Then, the results are combined as the stack unwinds back to the original call.

4. Recursion vs Iteration

Recursion and iteration (loops) can often solve the same problems. Recursion provides elegant solutions for problems with recursive structure (like trees), while iteration is generally more memory-efficient.

| Aspect | Recursion | Iteration |

| Code | Often simpler | Can be more verbose |

| Memory | Uses call stack | Uses less memory |

| Speed | Can be slower | Generally faster |

| Risk | Stack overflow | Infinite loops |

5. Fibonacci Sequence

The Fibonacci sequence is a series where each number is the sum of the two preceding ones: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Mathematically: $F(0) = 0$, $F(1) = 1$, $F(n) = F(n-1) + F(n-2)$ for $n > 1$

This is naturally expressed recursively since each Fibonacci number depends on the two previous ones.

6. Built-in Functions Review

Python provides many useful built-in functions that complement recursive programming:

• len() - Returns the length of a sequence

• max() - Returns the maximum of given values

• min() - Returns the minimum of given values

• type() - Returns the type of an object

• range() - Generates a sequence of numbers

7. Nested Loops for Tables

Nested loops are essential for generating tables and matrices. The outer loop controls rows and the inner loop controls columns:

```
for i in range(1, 11):
    for j in range(1, 11):
        print(i * j, end="\t")
    print()
```

8. String Alignment with print()

The print() function's end parameter can be used to control spacing. Using \t (tab) as the separator aligns output in columns:

```
print(value, end="\t")  # Prints with tab instead of newline
```

9. Recursion Depth Limit

Python has a default recursion depth limit (usually 1000) to prevent stack overflow. This can be checked and modified using sys.getrecursionlimit() and sys.setrecursionlimit().

10. When to Use Recursion

Recursion is ideal for problems with:

• Recursive mathematical definitions (factorial, Fibonacci)

• Tree and graph traversals

• Divide-and-conquer algorithms

• Problems that can be broken into similar subproblems

Lab Work:

Task 1: Fibonacci Number Using Recursion

Write a recursive function to calculate the Fibonacci number at a given position.

Code Summary:

This is the classic recursive Fibonacci implementation. The base cases handle positions 0 and 1, while the recursive case sums the two preceding Fibonacci numbers.

Key Code Lines:

```
# Fibonacci Using Recursion
def fib(n):
    if n == 0 or n == 1:
        return n
    return fib(n-1) + fib(n-2)

num = int(input("Enter a number: "))
print("Fibonacci at position", num, "is:", fib(num))
```

Output:

Task 2: Factorial Using Recursion

Write a recursive function to calculate the factorial of a number.

Code Summary:

Factorial is a natural fit for recursion: n! = n * (n-1)!. The base case is 0! = 1, and the recursive case multiplies n by the factorial of n-1.

Key Code Lines:

```
# Factorial Using Recursion
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

num = int(input("Enter a number: "))
print("Factorial of", num, "is:", factorial(num))
```

Output:

Task 3: Countdown Using Recursion

Write a recursive function that prints numbers counting down from a given value to 1.

Code Summary:

This task demonstrates recursion without accumulating a return value: the function prints the current number, then calls itself with a decremented value until the base case (0) is reached.

Key Code Lines:

```
# Task 3: Countdown Using Recursion
def show(n):
    if n == 0:
        return
    print(n)
    show(n - 1)

show(5)
```

Output:

Conclusions:

• Successfully understood the concept and mechanics of recursion

• Learned the two essential components: base case and recursive case

• Differentiated between recursive and iterative approaches

• Implemented recursive solutions for Fibonacci and factorial problems

• Understood how the call stack works during recursive execution

• Reviewed and applied built-in functions alongside user-defined functions

• Applied nested loops for generating formatted tables and patterns